Midas USTB Audit

Côme du Crest

2025-06-01

Table of contents

- Table of contents
- Midas USTB Audit
  - Scope
  - Context
  - Status
  - Legal Information And Disclaimer
- Issues
  - [Med] Redeeming USDC incurs a fee paid by the protocol
  - [Info] Redeemed token amount is truncated after min amount check

Midas USTB Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Midas Protocol.

For any inquiry contact me by email at come.ducrest@gmail.com or via telegram @ComeduCrest.

Scope

The scope includes the `RedemptionVaultWithUSTB` contract created by `PR 38` of repo `midas-apps/contracts`. The last reviewed commit of the PR is `0xc36f565`.

The code has been updated to fix founds issues in `PR 44`. The last viewed commit is `0x24835b4`.

Context

This audit follows a previous complete audit of the protocol dated `2024-08-09`. The `RedemptionVaultWithUSTB` inherits the audited `RedemptionVault`. It additionally enables the user to redeem mTokens for USDC directly by converting USTB into USDC using the USTB redemption contract.

The mainnet address of the USTB token is expected to be 0x43415eB6ff9DB7E26A15b704e7A3eDCe97d31C4e and the USTB redeem contract is expected to be 0x4c21B7577C8FE8b0B0669165ee7C8f67fa1454Cf.

Status

The report has been sent to the core developer.

All issues have been fixed in `PR 44`.

Legal Information And Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

Issues

[Med] Redeeming USDC incurs a fee paid by the protocol

Summary

When user redeems USDC the underlying USTB is converted to USDC by the USTB redeem contract which may incur a fee. This fee is paid by the vault and not the user due to the way redemption is handled.

Vulnerability Detail

The function `_checkAndRedeemUSTB()` checks if the vault has enough balance to send out the amount of tokens. If not it will redeem the missing amount by calculating the amount of USTB to redeem before redeeming it:

```
    function _checkAndRedeemUSTB(address tokenOut, uint256 amountTokenOut)
        internal
    {
        uint256 contractBalanceTokenOut = IERC20(tokenOut).balanceOf(
            address(this)
        );
        if (contractBalanceTokenOut >= amountTokenOut) return;

        require(tokenOut == ustbRedemption.USDC(), "RVU: invalid token");

        uint256 missingAmount = amountTokenOut - contractBalanceTokenOut;
        (uint256 ustbToRedeem, ) = ustbRedemption.calculateUstbIn(
             missingAmount
        );

        IERC20 ustb = IERC20(ustbRedemption.SUPERSTATE_TOKEN());
        uint256 ustbBalance = ustb.balanceOf(address(this));

        require(ustbBalance >= ustbToRedeem, "RVU: insufficient USTB balance");

        ustb.safeIncreaseAllowance(address(ustbRedemption), ustbToRedeem);
        ustbRedemption.redeem(ustbToRedeem);
    }
```

The redeem function uses `_calculateUsdcOut()` to compute the amount of USDC to send back to the caller:

```
    function redeem(uint256 superstateTokenInAmount) external {
        _redeem(msg.sender, superstateTokenInAmount);
    }

```

```
    function _redeem(address to, uint256 superstateTokenInAmount) internal
        override {
        _requireNotPaused();

        (uint256 usdcOutAmountAfterFee, uint256 usdcOutAmountBeforeFee, ) =
            _calculateUsdcOut(superstateTokenInAmount);

        if (USDC.balanceOf(address(this)) < usdcOutAmountAfterFee) revert
            InsufficientBalance();

        SUPERSTATE_TOKEN.safeTransferFrom({from: msg.sender, to: address(this),
             value: superstateTokenInAmount});
        USDC.safeTransfer({to: to, value: usdcOutAmountAfterFee});
        ISuperstateToken(address(SUPERSTATE_TOKEN)).offchainRedeem(
            superstateTokenInAmount);

        ...
    }
```

The function `_calculateUsdcOut()` may incur a fee if the `redemptionFee` admin controlled value is not `0`:

```
    function _calculateUsdcOut(uint256 superstateTokenInAmount)
        internal
        view
        returns (uint256 usdcOutAmountAfterFee, uint256 usdcOutAmountBeforeFee,
             uint256 usdPerUstbChainlinkRaw)
    {
        if (superstateTokenInAmount == 0) revert BadArgs();

        (bool isBadData,, uint256 usdPerUstbChainlinkRaw_) = _getChainlinkPrice
            ();
        if (isBadData) revert BadChainlinkData();

        usdPerUstbChainlinkRaw = usdPerUstbChainlinkRaw_;

        usdcOutAmountBeforeFee = (superstateTokenInAmount *
            usdPerUstbChainlinkRaw * USDC_PRECISION)
            / (CHAINLINK_FEED_PRECISION * SUPERSTATE_TOKEN_PRECISION);

        usdcOutAmountAfterFee = usdcOutAmountBeforeFee - calculateFee(
            usdcOutAmountBeforeFee);
    }

    function calculateFee(uint256 amount) public view returns (uint256) {
        return (amount * redemptionFee) / FEE_DENOMINATOR;
    }
```

The vault calculates how many USTB it needs to redeem to receive the exact amount of USDC it wants to send out. The fee increase the amount of USTB it needs to spend, resulting in a loss to the protocol.

Impact

USTB redeem fees are paid by the vault resulting in a guaranteed loss to the protocol if they are non-zero.

If deposit/redeem into the vault can be automated, attackers can loop deposits and redeem to force the vault to pay out fees multiple times. This can notably be abused by the fee recipient of the USTB redemption mechanism.

The fee parameter is currently set to 0 on the USTB redemption contract but can be changed at any time by the admin.

Code Snippets

https://github.com/midas-apps/contracts/blob/c36f5650c835b6eece797fdecdee6e2d28f7ac3c/contracts/RedemptionVaultWithUSTB.sol#L158-L180

https://github.com/superstateinc/onchain-redemptions/blob/59534af26d209ff5d3da5fda311c3a98471c1e71/src/RedemptionIdle.sol#L50-L68

https://github.com/superstateinc/onchain-redemptions/blob/59534af26d209ff5d3da5fda311c3a98471c1e71/src/Redemption.sol#L247-L263

https://github.com/superstateinc/onchain-redemptions/blob/59534af26d209ff5d3da5fda311c3a98471c1e71/src/Redemption.sol#L110-L112

Recommendation

Instead of calculating how many USDC the vault wants to send to the user and ensuring it has enough USDC by converting USTB to USDC it should calculate how many USTB it wants to send to the user and convert that exact amount to USDC. It should then enforce the resulting amount of USDC is above the `minReceiveAmount` specified by the user.

Response

The issue has been fixed in PR 45 by requiring the fees implemented by external redemption mechanism are zero.

[Info] Redeemed token amount is truncated after min amount check

Summary

When redeeming tokens, the user specifies a `minReceiveAmount` to make sure they receive the expected amount of tokens. In the code, the computed amount of tokens to send is required to be above this min value. However, the computed value is then truncated before being sent. It could be that the sent amount of token is below the `minReceiveAmount` value.

Vulnerability Detail

`redeemInstant()` enforces the computed `amountTokenOutWithoutFee` is greater than the user provided value for the min amount to receive. However, it then transfer out a value that is converted to token decimals and back to 18 decimals equivalent to a truncation:

```
    function redeemInstant(
        address tokenOut,
        uint256 amountMTokenIn,
        uint256 minReceiveAmount
    )
        external
        override
        whenFnNotPaused(this.redeemInstant.selector)
        onlyGreenlisted(msg.sender)
        onlyNotBlacklisted(msg.sender)
        onlyNotSanctioned(msg.sender)
    {
        ...

        require(
            amountTokenOutWithoutFee >= minReceiveAmountCopy,  // @audit-info
                minReceiveAmountCopy is 18 dp
            "RVU: minReceiveAmount > actual"
        );

        uint256 amountTokenOutWithoutFeeFrom18 = amountTokenOutWithoutFee
            .convertFromBase18(tokenDecimals);  // @audit-info loses precision
                if tokenDecimals < 18

        _checkAndRedeemUSTB(tokenOutCopy, amountTokenOutWithoutFeeFrom18);

        _tokenTransferToUser(
            tokenOutCopy,
            user,
            amountTokenOutWithoutFeeFrom18.convertToBase18(tokenDecimals),
            tokenDecimals
        );
        ...
    }
```

Impact

The `minReceiveAmount` is expected to be 18 decimals while the handled token could have less decimals. For example for a token with 17 decimals if `minReceiveAmount` is `1e18 + 1`, it will be truncated to `1e18` and resulting in the user receiving an amount of tokens that when scaled back to 18 decimals is less than `1e18`.

I do not think uses should expect to receive `1e18 + 1` tokens for a token with 17 decimals so the impact is very low but it does not cost much to fix it.

Code Snippets

https://github.com/midas-apps/contracts/blob/c36f5650c835b6eece797fdecdee6e2d28f7ac3c/contracts/RedemptionVaultWithUSTB.sol#L126-L141

Recommendation

Enforce the sent token amount is greater than expected min value after truncation.

Response

The issue has been fixed in PR 43 by truncating the sent token amount before using it in the require.